# Fast Gaussian process regression via low-rank approximations

Junhyung Chang jc10314@nyu.edu
Shrikant Chand sc7075@nyu.edu
New York University

**Abstract**

In this project, we explore three methods for large-scale Gaussian process regression. First, a single-pass randomized SVD [6] is implemented to construct a global low-rank approximation of the Gram matrix $K(\mathbf{x}, \mathbf{x})$. The second method is to compute the Karhunen-Loéve (KL) expansion, which also allows one to construct a global low-rank approximation of the Gram matrix. The final method involves exploiting the hierarchical off-diagonal low-rank structure of the Gram matrix to build a multi-level HODLR factorization to rapidly invert the Gram matrix. The RSVD scales quadratically with the system size ($O(n^2 r)$), which makes it uncompetitive in a sequential setting. However, it is highly parallelizable, and there exist ways to accelerate the method. The KL-expansion approach scales linearly with system size ($O(nm^2)$), but can potentially introduce discretization and truncation errors. The HODLR factorization is a near-linear time algorithm ($O(n \log^2 n)$), which can be combined with quick low-rank approximation schemes to cut back the run-time. Numerical experiments are conducted to compare the three methods on the task of computing the posterior mean of a Gaussian process.

## Contents

**1 Introduction** **2**

**2 Randomized low-rank approximations** **4**
2.1 A brief overview . . . . . 4
2.2 Randomized SVD . . . . . 4
2.3 A single-pass algorithm for the randomized eigenvalue decomposition 5
2.4 Gaussian process regression via randomized approximate eigenvalue decomposition . . . . . 6

**3 Gaussian process regression via eigenfunction expansion** **6**
3.1 Computing the Karhunen-Loéve expansion . . . . . 6
3.2 GP regression via KL-expansion . . . . . 7

**4 Gaussian process regression via HODLR matrix factorization** **8**
4.1 HODLR decomposition . . . . . 8
4.2 Low-rank approximation of the off-diagonal blocks . . . . . 10

**5 Numerical experiments** **11**

**6 Conclusion** **12**

**References** **13**

# 1 Introduction

A Gaussian process is completely characterized by a kernel, also known as a covariance function, and a mean function. The covariance function takes two input data, and returns the covariance of the output at the two input data points. Explicitly, for two sets of of input-output data points $(x_k, y_k)_{k=1,2}$, a covariance function takes the form

$$k(x_1, x_2) = \mathrm{cov}(y_1, y_2) = \mathbb{E}[(y_1 - m(x_1))(y_2 - m(x_2))],$$

where $m(x)$ is the mean function defined as

$$m(x_i) = \mathbb{E}[y_i].$$

For vectors $\mathbf{x} \in \mathbb{R}^m$, and $\mathbf{x}' \in \mathbb{R}^n$, the Gram matrix is defined as

$$K(\mathbf{x}, \mathbf{x}') = (k(\mathbf{x}_i, \mathbf{x}_j))_{ij} \in \mathbb{R}^{m\times n}.$$

Since a Gaussian process is completely defined by the mean and covariance functions, the usual notation for a Gaussian process is

$$f(x) = \mathcal{GP}(m(x), k(x, x')).$$

A descriptive list of widely used covariance functions is included in [9, ch.4]. Note that Gaussian processes do not require any other assumptions such as the underlying shape of the data, which makes Gaussian processes a highly flexible framework for data analysis.

Once the mean and covariance functions are specified, Gaussian process regression allows one to make predictive statements about the output of test points $\mathbf{x}^* = (x_i^*)_{i=1,\dots,m}$, by viewing the output $\mathbf{y}^* = (y_i^*)_{i=1,\dots,m}$ as a realization of a normal random vector whose mean and covariance are determined by conditioning $\mathbf{y}^*$ on the training data $(\mathbf{x}, \mathbf{y}) = (x_i, y_i)_{i=1,\dots,n}$, and the test input $\mathbf{x}^*$. This requires the use of several Gaussian identities introduced in [9, Appendix A].

To summarize the results, suppose the data is produced by the following functional relation

$$y_i = f(x_i) + \varepsilon_i,$$

where $\varepsilon_i \sim \mathcal{N}(0, \sigma_i^2)$ is independent white noise. In the noise-free case, one can

simply set $\sigma_i = 0$ for all $i$, and view $\varepsilon_i \equiv 0$.

The prior distribution can be written as

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{y}^* \end{bmatrix} \sim \mathcal{N}\left( \begin{bmatrix} m(\mathbf{x}) \\ m(\mathbf{x}^*) \end{bmatrix}, \begin{bmatrix} K(\mathbf{x},\mathbf{x}) + \sigma^2 I & K(\mathbf{x},\mathbf{x}^*) \\ K(\mathbf{x}^*,\mathbf{x}) & K(\mathbf{x}^*,\mathbf{x}^*) \end{bmatrix} \right), \tag{1.1}$$

where $\sigma^2$ is a length-$n$ row vector containing the variances $\sigma_i^2$. In the noise-free case, the entries of $\sigma^2$ are all zeros. For convenience, we assume in this project that all the entries of $\sigma^2$ are one, so that the $\varepsilon_i$'s are independent standard white noise.

It is shown in [9] that the posterior distribution of the outputs $\mathbf{y}^*$ conditioned on $\mathbf{x}, \mathbf{y}, \mathbf{x}^*$ is

$$\mathbf{y}^* | \mathbf{x}, \mathbf{y}, \mathbf{x}^* \sim \mathcal{N}(\overline{\mathbf{y}}^*, \mathrm{cov}(\mathbf{y}^*)) \tag{1.2}$$

where

$$\overline{\mathbf{y}}^* = m(\mathbf{x}^*) + K(\mathbf{x}^*,\mathbf{x})[K(\mathbf{x},\mathbf{x}) + \sigma^2 I]^{-1}(\mathbf{y} - m(\mathbf{x})), \tag{1.3}$$

$$\mathrm{cov}(\mathbf{f}^*) = K(\mathbf{x}^*,\mathbf{x}^*) - K(\mathbf{x}^*,\mathbf{x})[K(\mathbf{x},\mathbf{x}) + \sigma^2 I]^{-1} K(\mathbf{x},\mathbf{x}^*). \tag{1.4}$$

Henceforth in this project, we will let $m(x) \equiv 0$ for convenience.

As can be seen in equations (1.3), (1.4), the main computational bottleneck of Gaussian process regression is computing $[\sigma^2 I + K(\mathbf{x},\mathbf{x})]^{-1}$. For many covariance functions, $K(\mathbf{x},\mathbf{x})$ is highly low-rank when $n$ is large. However, When the data is known to contain noise, $\sigma^2 I + K(\mathbf{x},\mathbf{x})$ is full rank, thus one faces the problem of inverting a dense matrix, which is an $O(n^3)$ operation. Nevertheless, there are several ways to circumvent this issue. First, a global low-rank approximation of the Gram matrix $K(\mathbf{x},\mathbf{x})$ can be computed using linear algebra. By using a randomized-scheme such as in [6], [10], a low-rank approximation can be computed in $O(n^2 \log m)$ time, where $n$ is the number of training inputs, and $m$ is the approximate rank of $K(\mathbf{x},\mathbf{x})$. Secondly, it is shown in [3] that by taking a low-order Karhunen-Loéve (KL) expansion of the Gaussian process via an eigenfunction expansion, one can efficiently compute a global low-rank approximation of the Gram matrix in $O(nm^2)$ time, where $n$ is the number of observations, and $m$ is the length of the low-order eigenfunction expansion. Numerical experiments support this computational advantage of the eigenfunction approach for large system sizes, typically greater than $O(10^5)$. The third approach, which is shown in [2], uses the idea that for certain classes of covariance functions, the matrix $\sigma^2 I + K(\mathbf{x},\mathbf{x})$ meets the Hierarchical Off-Diagonal Low-Rank (HODLR) conditions introduced in e.g. [1]. This condition can be exploited to compress the matrix, and reduce the cost of computing the inverse to $O(n \log^2 n)$.

While traditional deterministic methods for low-rank approximations are well studied, recent results such as [6] demonstrate the superiority of randomized methods, especially for large scale matrix computations. The following section introduces the main idea of randomized methods, along with examples of randomized matrix factorizations that can be directly used in Gaussian process regression.

# 2 Randomized low-rank approximations

## 2.1 A brief overview

Without loss of generality, let $K \in \mathbb{C}^{m\times n}$, $m \leq n$ be any matrix of approximate rank $r < m$. The key idea of any randomized approximate factorization is to find an approximate orthonormal basis $Q$ for the range of $K$ using a randomized scheme. Once $Q$ is found, then $K$ can be approximated by the projection of $K$ onto the approximate range spanned by $Q$, namely

$$K \approx QQ^*K. \tag{2.1}$$

From the expression (2.1), one can form various low-rank factorizations of $K$ such as the SVD, QR factorization, interpolative decomposition, etc., via standard deterministic linear algebra routines. Later in this project, we review a simple algorithm introduced in [4], [6] which manipulates (2.1) to form an approximate SVD of $K$.

The main computational hurdle of randomized methods is finding the approximate basis $Q$, which the authors of [4], [6] call "Stage A". The idea is to draw a random matrix $\Omega \in \mathbb{C}^{m\times r}$, and form

$$Y = K\Omega. \tag{2.2}$$

In essence, each column of $Y$ is in the range of $K$, and the columns will be linearly independent almost surely, due to the random structure of $\Omega$. One can further orthonormalize the columns of $Y$ via e.g. Gram-Schmidt to form the approximate orthonormal basis $Q$.

It is clear that the main computational challenge is the matrix product $K\Omega$ in (2.2), which costs $O(mnr)$ flops. However, this can be easily overcome by for example, drawing one row of $\Omega$ at a time and performing the vector-matrix product $\Omega(j,:)K$ in parallel. Using this method squentially, the randomized approximate SVD of a $10^5 \times 10^5$ matrix with $r \approx 15$ was computed within four minutes on a standard laptop without using GPU. The time will be reduced drastically if implemented in parallel on a GPU.

Furthermore, it also turns out that the distribution of $\Omega$ can be carefully chosen to accelerate the matrix product $K\Omega$. If one simply draws the entries of $\Omega$ from a standard normal distribution, the matrix product costs $O(mnr)$ flops. However, this can be reduced to $O(mn \log r)$ flops when $\Omega$ is sufficiently structured, for example, by using the Fast Johnson-Lindenstrauss transform [12].

## 2.2 Randomized SVD

Once the approximate basis $Q$ is computed, the matrix $K$ can be approximated in the form (2.1). If one takes the SVD of $Q^*K$, namely $Q^*K = \hat{U}SV^*$, then (2.1) can be re-written as

$$K \approx Q\hat{U}SV^* = USV^*. \tag{2.3}$$

It is clear that $U = Q\hat{U}$ is unitary. This processes is named "Stage B" in [4], [6].

From (2.1), one can also find a generic low-rank approximation

$$K \approx \tilde{U}\tilde{V}^*, \tag{2.4}$$

where $\tilde{U} \in \mathbb{C}^{m\times r}$, and $\tilde{V} \in \mathbb{C}^{n\times r}$. This can be done e.g., by taking $\tilde{U} = Q$, and $\tilde{V}^* = Q^*K$. The approximation (2.4) will be used to compress the off-diagonal low-rank blocks in the HODLR factorization in section 4.

## 2.3 A single-pass algorithm for the randomized eigenvalue decomposition

While deriving (2.3), one needs to form the matrix $Q^*K$, and compute its SVD. This is the second time that the full matrix $K$ is accessed because $K$ was already used once during "Stage A", while forming $Q$. The second call for $K$ can be avoided by using a method introduced in [4], [6]. There are two versions to this method in [4], [6], for Hermitian and non-Hermitian matrices respectively. In this project, the matrix $K$ will always be Hermitian since $K$ is a Gram matrix. Therefore, only the Hermitian single-pass algorithm is required.

Observing (2.1), one can take the complex transpose, and use the fact that $K$ is Hermitian to write

$$KQQ^* \approx K \approx QQ^*K. \tag{2.5}$$

From (2.5), one obtains

$$K \approx QQ^*KQQ^*. \tag{2.6}$$

Let

$$C = Q^*KQ, \tag{2.7}$$

and take its eigenvalue decomposition $C = \hat{U}D\hat{U}^*$. Then, the desired approximate decomposition of $K$ is

$$K \approx Q\hat{U}D\hat{U}^*Q^* = UDU^*, \tag{2.8}$$

where $U = Q\hat{U}$.

Since the goal is not to use $K$ in the computation of $C$ in (2.7), one can alternatively multiply the matrix $Q^*\Omega \in \mathbb{C}^{r\times r}$ to the right of (2.7) to get

$$C(Q^*\Omega) = Q^*KQQ^*\Omega. \tag{2.9}$$

Since $K \approx KQQ^*$, and $Y = K\Omega$ is already computed in equation (2.2) during "Stage A", one can re-write (2.9) as

$$C(Q^*\Omega) \approx Q^*K\Omega = Q^*Y. \tag{2.10}$$

Thus $C$ can be computed without using $K$ by solving the system (2.10), which can be done quickly, since $C$, $Q^*\Omega$, and $Q^*Y$ are all $r \times r$ matrices.

This method will speed up the computation of the eigenvalue decomposition, but is less accurate than the double-pass algorithm in section 2.2.

In practice, for all of the methods above, one typically over-samples $r$ by adding

an over-sampling parameter $p$ to $r$ for best results.

### 2.4 Gaussian process regression via randomized approximate eigenvalue decomposition

If $K(\mathbf{x}, \mathbf{x}) \approx UDU^*$ as in (2.8), the task of computing

$$[K(\mathbf{x}, \mathbf{x}) + I_n]^{-1} \tag{2.11}$$

can be written as

$$\begin{aligned} [UDU^* + I_n]^{-1} &= [UDU^* + UI_rU^*]^{-1} \\ &= U[D + I_r]^{-1}U^*. \end{aligned} \tag{2.12}$$

Since $D + I_r$ is a $r \times r$ diagonal matrix, the computation of its inverse is particularly simple.

The main computational challenge is finding the approximate eigenvalue decomposition (2.8), and more specifically, the matrix multiplication involving the full matrix $K$ in (2.2). As mentioned in section 2.1, equation (2.2) can be easily parallelized, and there exist methods to further accelerate (2.2).

## 3 Gaussian process regression via eigenfunction expansion

### 3.1 Computing the Karhunen-Loéve expansion

Another approach to finding a global low-rank approximation of $K$ involves computing a low-order eigenfunction expansion of the integral operator $\mathcal{K} : L^2[-1, 1] \to L^2[-1, 1]$ defined by

$$\mathcal{K}f(x) = \int_{-1}^{1} k(x, x')f(x')dx'. \tag{3.1}$$

The interval $[-1, 1]$ was chosen to make use of the canonical Gauss-Legendre quadrature nodes and weights, but the Gauss nodes can be scaled to incorporate any interval, for example $[-3, 3]$, which is used for the experiments in section 5. Furthermore, one could generalize this operator to be on square integrable functions on a $d$-dimensional region $D$. In this project, we focus on the case $D = [-1, 1]$. If $u(x)$ is an eigenfunction of $\mathcal{K}$, and $\lambda$ is the corresponding eigenvalue, then

$$\lambda u(x) = \mathcal{K}u(x) = \int_{-1}^{1} k(x, x')u(x')dx'. \tag{3.2}$$

The key observation in [3] is that if $f$ is a zero mean Gaussian process on $D$ with

covariance function $k$, then any order-$m$ approximation

$$\hat{f}(x) = \sum_{i=1}^{m} \alpha_i g_i(x), \tag{3.3}$$

where $\alpha_i \sim \mathcal{N}(0,1)$, has covariance function

$$\begin{aligned}
\hat{k}(x,x') &= \mathbb{E}[\hat{f}(x)\hat{f}(x')] \\
&= \mathbb{E}\left[\left(\sum_{i=1}^{m} \alpha_i g_i(x)\right)\left(\sum_{i=1}^{m} \alpha_i g_i(x')\right)\right] \\
&= \sum_{i,j=1}^{m} g_i(x) g_j(x') \mathbb{E}[\alpha_i \alpha_j] \\
&= \sum_{i=1}^{m} g_i(x) g_i(x').
\end{aligned} \tag{3.4}$$

Moreover, it is shown in [11] that if the $g_i(x)$'s are chosen to be the basis functions of the Karhunen-Loéve (KL) expansion, namely $\varphi_i(x) := \sqrt{\lambda_i} u_i(x)$, where the $u_i(x)$'s are the normalized eigenfunctions of $\mathcal{K}$ corresponding to the eigenvalues $\lambda_i$'s, which are in decreasing order, then the covariance function $\hat{k}$ of $\hat{f}$ is the best approximation of the covariance function $k$ of $f$ in the $L^2$ norm. Mercer's theorem stated in e.g., [3] guarantees that

$$\sum_{i=1}^{r} \varphi_i(x)\varphi_i(x') \to k(x,x') \tag{3.5}$$

absolutely, and uniformly.

There are standard methods of finding the eigenvalues and eigenfunctions of $\mathcal{K}$, which involve a discretization matrix $A$ that is obtained by discretizing $\mathcal{K}$ at the $r$-th order Gaussian nodes. Then, the eigenvalues of $A$ are the first $r$ eigenvalues of $\mathcal{K}$, and the eigenvectors of $A$ are viewed as the eigenfunctions evaluated at each of the Gaussian nodes. A straightforward implementation is introduced in [3, Algorithm 1], which is summarized in algorithm 3.1.

This is an $O(nr^2)$ algorithm on the interval, so it scales linearly with the system size. However, this method suffers from the curse of dimensionality as do similar discretization methods. Also, one must be aware of the additional error added in the discretization process, as well as the truncation error. It is noticed in the experiments in 5 that one needs to increase the length of the KL-expansion when the system size becomes larger, regardless of the true rank of the matrix. For example, for smaller matrices, $m = 15$ was sufficient to achieve a desirable precision, but for system size $10^6$, one needs at least $m = 25$.

## 3.2 GP regression via KL-expansion

Once the order-$m$ KL basis functions $(\varphi_i(x))_{i=1,\dots,m}$ are obtained, these can be used to form a low-rank approximation of the Gram matrix $K$. By (3.5), each entry in

**Algorithm 1** Computing the KL-expansion

1: Form $n \times n$ discretization matrix $A$ defined by $A_{i,j} = \sqrt{w_i w_j} k(x_i, x_j)$, where $x_i, w_i$'s are the order-$r$ Gauss nodes and weights.
2: Compute eigenvalue decomposition of $A$: $A = UDU^*$. The eigenvalues are on the diagonals of $D$
3: Scale the rows of $U$ to form matrix of eigenfunctions evaluated at Gaussian nodes: $\hat{U}_{i,:} = U_{i,:}/\sqrt{w_i}$.
4: Use length-$r$ Legendre polynomial approximation to find the eigenfunctions based on the columns of $\hat{U}$. Store the coefficients of the Legendre approximation in matrix $A = M\hat{U}$, where $M$ is the matrix that transforms the evaluated function values into the coefficients.
5: The eigenfunctions are $u_i(x) = \sum_{j=1}^{r} A_{j,i} P_{j-1}(x)$ on $x \in [-1, 1]$, where $P_j(x)$ is the $j$-th order Legendre polynomial.
6: Set $\varphi_i(x) = \sqrt{\lambda_i} u_i(x)$.

$K$ can be approximated in the form

$$k(x, x') \approx \sum_{i=1}^{r} \varphi_i(x)\varphi_i(x'). \tag{3.6}$$

Defining $X := (\varphi_j(x_i))_{ij}$, then $XX^*$ is a natural low-rank approximation of $K$, since each entry of $XX^*$ is precisely the right hand side of (3.6). Hence, $[I+K]^{-1}$ in (1.3) is approximated by

$$[I + XX^*]^{-1}. \tag{3.7}$$

By computing the SVD $X = UDV^*$, then (3.7) can be written as

$$[I + UD^2U^*]^{-1}. \tag{3.8}$$

One can apply the same technique used in (2.12) to compute the inverse quickly via

$$U[I_r + D^2]^{-1}U^*. \tag{3.9}$$

# 4 Gaussian process regression via HODLR matrix factorization

## 4.1 HODLR decomposition

Many covariance functions give rise to a Gram matrix $K$ for which $A := I + K$ satisfies the Hierarchical Off-Diagonal Low-Rank (HODLR) condition, which is defined in [7] as the following:

**Definition 4.1** (HODLR Matrix)**.** Let $A$ be a matrix of size $n \times n$, and let $k$ be an integer such that $k < n$. We say that $A$ is a HODLR matrix with rank $k$ if either of the following conditions holds:

- $A$ is itself of size at most $2k \times 2k$.
- If $A$ is partitioned into four equal sized blocks,

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix},$$

  then $A_{12}$ and $A_{21}$ have rank at most $k$ and $A_{11}$ and $A_{22}$ are also HODLR matrices of rank $k$.

The off-diagonal low rank blocks can be compressed in the form

$$A_{ij} = UV^*,$$

where $U, V \in \mathbb{C}^{n\times\ell}$, and $\ell$ is typically the target rank that is determined by the desired precision. Hence, the matrix $A$ is approximated as

$$A \approx \begin{pmatrix} A_{11} & UV^* \\ VU^* & A_{22} \end{pmatrix},$$

which can be viewed as a low-rank update to a block diagonal matrix:

$$A \approx \begin{pmatrix} A_{11} & 0 \\ 0 & A_{22} \end{pmatrix} + \begin{pmatrix} 0 & U \\ V & 0 \end{pmatrix} \begin{pmatrix} U^* & 0 \\ 0 & V^* \end{pmatrix}. \tag{4.1}$$

The inverse of matrices in this form can be efficiently computed via the Woodbury matrix identity.

**Definition 4.2** (Woodbury Matrix Identity)**.**

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1},$$

where $A, C$ are full rank, and $U, V$ are rectangular matrices. In the case $C = I$, and $A = I$ (in respective sizes), the formula becomes

$$(I + UV)^{-1} = I - U(I + VU)^{-1}V.$$

If one decides to further compress the off diagonal blocks of $A_{11}$, and $A_{22}$, an effective way to invert the multi-level compressed matrix is to construct a HODLR factorization, which can be done by recursively factoring out the block diagonals from the previous level. For example, the first level of this factorization is

$$\begin{aligned} A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} &= \begin{pmatrix} A_{11} & 0 \\ 0 & A_{22} \end{pmatrix} \begin{pmatrix} I & A_{11}^{-1}A_{12} \\ A_{22}^{-1}A_{21} & I \end{pmatrix} \\ &\approx \begin{pmatrix} A_{11} & 0 \\ 0 & A_{22} \end{pmatrix} \begin{pmatrix} I & A_{11}^{-1}UV^* \\ A_{22}^{-1}VU^* & I \end{pmatrix}. \end{aligned} \tag{4.2}$$

One can repeat this procedure to obtain a two-level factorization by splitting the diagonal blocks $A_{11}$ and $A_{22}$ into four smaller blocks, then factoring out the four

smaller diagonal blocks. In the end, the matrix $A$ is decomposed into a chain of factors

$$A = A_\kappa A_k A_{k-1} \dots A_1, \tag{4.3}$$

and the inverse of $A$ is

$$A^{-1} = A_1^{-1} \dots A_{k-1}^{-1} A_k^{-1} A_\kappa^{-1}. \tag{4.4}$$

In the whole process, the only matrices that need to be stored are the low-rank factors $U_i^{(j)}, V_i^{(j)}$. For example, the factor $A_1$ looks like

$$\begin{pmatrix} I & A_{11}^{-1} U V^* \\ A_{22}^{-1} V U^* & I \end{pmatrix}.$$

The computation of $A_{11}^{-1}, A_{22}^{-1}$ require information from all the factors $A_2, \dots, A_\kappa$, but one cannot afford to store the inverse diagonal blocks, because they quickly become too large to store. It turns out that the inverses never even have to be formed explicitly if one uses the Woodbury matrix identity 4.2 using the low-rank factors. That is, instead of forming each inverse of the diagonal blocks, one applies the matrices occurring in the Woodbury formula to all the remaining $U_i^{(j)}, V_i^{(j)}$'s in the right locations, hence updating the low-rank matrices in the remaining factors of the HODLR factorization.

Since the goal is to compute the posterior mean, which requires

$$A^{-1}\mathbf{y} = A_1^{-1} \dots A_{k-1}^{-1} A_k^{-1} A_\kappa^{-1} \mathbf{y},$$

one must also recursively apply each inverse to $\mathbf{y}$ as soon as the inverse is computed via the Woodbury matrix identity. See [2] for detailed explanations. A pseudocode for computing the posterior mean of a Gaussian process via the HODLR factorization is in algorithm 2 below. The complexity for constructing the HODLR factorization is $O(n \log^2 n)$, but the actual run-time is dominated by computing the low-rank approximations of the off-diagonal blocks.

## 4.2 Low-rank approximation of the off-diagonal blocks

In this project, the machinery for the low-rank approximation of the off-diagonal blocks is either the low-rank factors obtained from the randomized SVD (2.4), or the KL-expansion in section 3. It was noted that the randomized SVD is very fast for small matrices, but scales badly with $n$, due to the $O(n^2 m)$ complexity. On the other hand, the KL-expansion approach is $O(nm^2)$, hence scales linearly with $n$, but is typically slower for smaller matrices. In a HODLR factorization, although the blocks that need to be approximated decrease in size as the levels are increased, the number of the blocks increase at the same rate. Therefore, solely using KL-expansion for all of the blocks will result in a longer run-time due to the smaller blocks. Consequently, a way to decrease the overall run-time is to use the KL-expansion for larger blocks, then switch to randomized SVD once the size of the blocks become sufficiently small. For this experiment, it is observed empirically that the randomized SVD becomes slower than the KL-expansion when the system size

**Algorithm 2** Computing posterior mean (1.3) via HODLR factorization

**Require:** $p$: Size of smallest diagonal block in $K_\kappa$
**Require:** $\epsilon$: Precision of low-rank approximation
**Require:** Matrix entry evaluation routine (e.g. kernel function)
**Require:** training data $(\mathbf{x}, \mathbf{y})$

```
κ ← ⌊log_2(n/p)⌋
for j = 1 to κ do
    for i = 1 to 2^j do
        Compute low rank factorizations U_i^j, V_i^j of all off diagonal blocks
    end for
end for
Form block diagonal matrix K_κ from block diagonals of K, then apply inverse
of each block to y, and every remaining U_i^(l) or V_i^(l) (l < κ) in the right place.
for j = κ to 1 do
    for i = 1 to 2^j do
        Compute inverse of i-th block in K_j via Woodbury matrix identity, and
apply to ỹ and every remaining U_i^(l) or V_i^(l) (l < j) in the right place. {ỹ is all
the previous inverse matrices applied to y}
    end for
end for
```

is greater than approximately $10^4$. Therefore, $10^4$ is a good point to switch between the two methods.

# 5 Numerical experiments

In this project, we focus on the task of computing the posterior mean (1.3) for a Gaussian process with inputs on the interval $[-3, 3]$. The time and error for each of the three methods are computed for system sizes up to $n = 10^6$. A 1.30 GHz Intel Core i7 processor CPU with 16 GB 3200 MHz RAM was used for the following experiments.

In a sequential setting, the global randomized low-rank scheme (method 1) becomes infeasible for system sizes larger than $n = 10^5$, but will be more competitive once equation (2.2) is implemented in parallel on a GPU.

The length of the KL-expansion is kept at $m = 25$ throughout the experiment, although it could be decreased without affecting the precision for smaller matrices. It can be seen that the time for computing the KL-expansion basis functions remain roughly equal. The only part that scales with system size is the time corresponding to the formation of the low-rank approximation, and inverting $I + XX^*$.

In the following experiment, the training inputs $\mathbf{x}$ were chosen randomly on the interval $[-3, 3]$, and the training outputs $\mathbf{y}$ were chosen randomly on the interval $[0, 2]$. The test inputs $\mathbf{x}^*$ were chosen as 100 equispaced points on the interval $[-3, 3]$. The Gaussian covariance function $k(x, x') = e^{-|x-x'|^2}$ is used. The following plots are the posterior mean functions evaluated at $\mathbf{x}^*$.

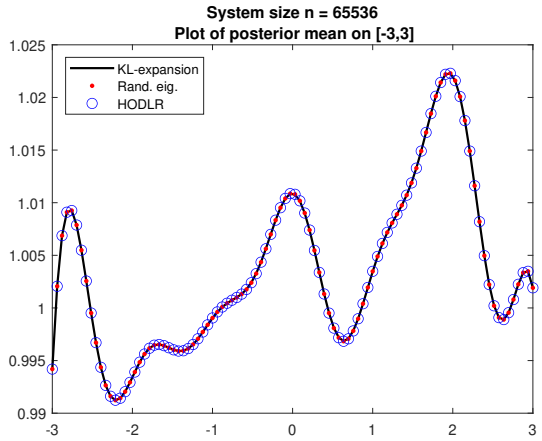


Figure 1: System size: $n = 2^{16}$

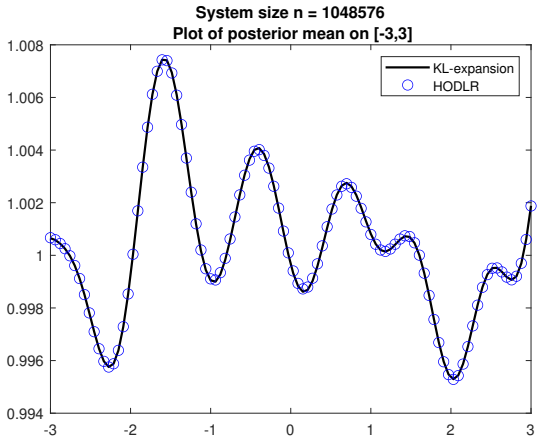


Figure 2: System size: $n = 2^{20}$

| $n$ | Stage A | Stage B | Invert |
|---|---|---|---|
| $2^8$ | 0.0117 | 0.0023 | 0.0022 |
| $2^{10}$ | 0.0727 | 0.0005 | 0.0018 |
| $2^{12}$ | 0.5718 | 0.0013 | 0.0004 |
| $2^{14}$ | 7.2198 | 0.0044 | 0.0012 |
| $2^{16}$ | 106.8086 | 0.0142 | 0.0072 |
| $2^{18} \approx 2 \times 10^5$ | 1974.8465 | 0.0436 | 0.0286 |

Figure 3: Run-time in seconds for method 1 (randomized SVD)

| $n$ | KL-expand | Factor & invert |
|---|---|---|
| $2^8$ | 1.1837 | 0.0810 |
| $2^{10}$ | 1.1125 | 0.3377 |
| $2^{12}$ | 0.9705 | 1.4091 |
| $2^{14}$ | 1.2947 | 6.1126 |
| $2^{16}$ | 0.9394 | 19.7492 |
| $2^{18}$ | 1.0667 | 79.3289 |
| $2^{20} \approx 10^6$ | 0.8980 | 332.6400 |

Figure 4: Run-time in seconds for method 2 (KL-expansion)

# 6 Conclusion

Solely using the RSVD to construct a global low-rank approximation is very fast for small matrices, but becomes relatively slow when the matrix size increases to above $10^4$. However, this method can be greatly accelerated by implementing the operation (2.2) in parallel. The KL-expansion approach was the fastest for large matrices, but one needs to pay attention to the additional discretization error, and may have to increase the length of the KL-expansion regardless of the true rank of the matrix. The HODLR factorization is feasible for system sizes $\sim 10^6$, but its run-time is highly

| $n$ | Compress | Invert |
|---|---|---|
| $2^8$ | 0.0236 | 0.0391 |
| $2^{10}$ | 0.0697 | 0.0378 |
| $2^{12}$ | 1.0019 | 0.2265 |
| $2^{14}$ | 8.0835 | 1.3945 |
| $2^{16}$ | 127.6205 | 5.7087 |
| $2^{18}$ | 581.8549 | 29.4437 |
| $2^{20} \approx 10^6$ | 2844.7266 | 150.3182 |

Figure 5: Run-time in seconds for method 3 (HODLR factorization)

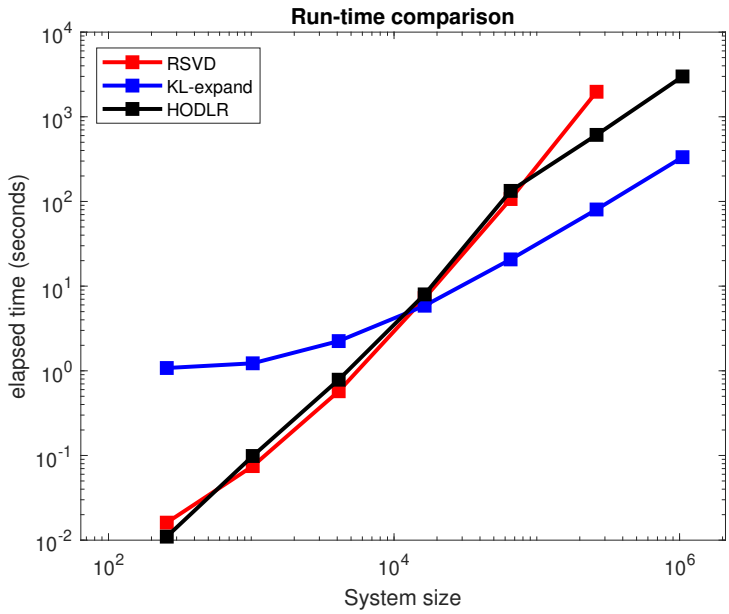


Figure 6: Run-time comparison of the three methods

dependent on the method used to compress the off-diagonal low-rank matrices. From figure 5, it can be noticed that when HODLR uses RSVD to approximate its off-diagonal matrices, it scales similar to RSVD. However, once the system size get large, and the KL-expansion is used to compress larger blocks, HODLR starts to scale like the KL-expansion approach. Thus, one could further improve the overall run-time of the HODLR factorization approach by using a quicker method to find a low-rank approximation for the off-diagonal blocks.

# References


[1] S. Ambikasaran and E. Darve. "An $O(n \log n)$ Fast Direct Solver for Partial Hierarchically Semi-Separable Matrices". In: *J. Sci. Comput.* 57.3 (2013), pp. 477–501.

[2] S. Ambikasaran et al. "Fast Direct Methods for Gaussian Processes". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 38.2 (2016), pp. 252–265. DOI: 10.1109/TPAMI.2015.2448083.

[3] P. Greengard and M. O'Neil. "Efficient reduced-rank methods for Gaussian processes with eigenfunction expansions". In: *arXiv pre-print* (2021). URL: https://arxiv.org/abs/2108.05924v1.

[4] N. Halko, P. G. Martinsson, and J. A. Tropp. "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions". In: *SIAM Review* 53.2 (2011), pp. 217–288. URL: https://arxiv.org/abs/0909.4061.

[5] E. Liberty et al. "Randomized algorithms for the low-rank approximation of matrices". In: *Proceedings of the National Academy of Sciences* 104.51 (2008), pp. 20167–72. DOI: 10.1073/pnas.0709640104.

[6] P. G. Martinsson. "Randomized methods for matrix computations". In: *IAS/Park City Mathematics Series* (2018). URL: https://arxiv.org/abs/1607.01649.

[7] P.-G. Martinsson. *Fast Direct Solvers for Elliptic PDEs*. SIAM, 2019. ISBN: 978-1-61197-603-8. DOI: https://doi.org/10.1137/1.9781611976045.

[8] P.-G. Martinsson, V. Rokhlin, and M. Tygert. "A randomized algorithm for the decomposition of matrices". In: *Applied Computational Harmonic Analysis* 30 (2011), pp. 47–68.

[9] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. the MIT Press, 2006. ISBN: 26218253X.

[10] V. Rokhlin, A. Szlam, and M. Tygert. "Randomized algorithms for the low-rank approximation of matrices". In: *SIAM J. Matrix Anal. Appl.* 31.3 (2009), pp. 1100–1124. DOI: https://doi.org/10.1137/080736417.

[11] L. N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, 2019. ISBN: 978-1-611975-93-2.

[12] F. Woolfe et al. "A fast randomized algorithm for the approximation of matrices". In: *Applied and Computational Harmonic Analysis* 25.3 (2008), pp. 335–366. DOI: https://doi.org/10.1016/j.acha.2007.12.002.